PHƯƠNG PHÁP KIỂM THỬ THEO FUZZING CHO HỆ THỐNG BÁN HÀNG

TRƯỜNG ĐẠI HỌC PHENIKAA

KHOA HỆ THỐNG THÔNG TIN

BÁO CÁO BÀI TẬP LỚN — CSE703093: AN TOÀN PHẦN MỀM

Giảng viên: ThS. Vũ Quang Dũng

Họ tên: [ĐIỀN HỌ TÊN]

Mã số sinh viên: [ĐIỀN MSSV]

Lớp: [ĐIỀN LỚP]

Học kỳ: [ĐIỀN HỌC KỲ]

Bản biên soạn có bằng chứng thực nghiệm; cần điền thông tin, kiểm tra số trang và xuất PDF trước khi nộp.

MỤC LỤC

1. Giới thiệu và phạm vi 4

Đề tài áp dụng fuzzing cho API bán hàng mô phỏng: blackbox khám phá input bất thường; coverage-guided chạy lại, khảo sát lân cận và chọn input mở thêm dòng; whitebox concolic có giới hạn dùng trace nhánh totalAmount và Z3 để sinh input, xác nhận qua API. Các phần được đánh giá riêng theo bằng chứng thực thi. 4

Năm module gồm đăng nhập, tạo đơn, trạng thái đơn, người dùng và sản phẩm. Oracle phát hiện vi phạm quyền, toàn vẹn dữ liệu, trạng thái và xử lý input. Bộ chạy API chưa thực thi timing, bộ đếm, concurrency/version, CTL hoặc các mẫu C/Python; SMT hiện dùng riêng trong concolic có chọn lọc, không xác minh toàn bộ mô hình Excel. 4

Bộ đặc tả giữ 10 sheet/file và 104 testcase: AUTH 19, ORD 21, OS 22, USR 21, PRD 21. ID được giữ nguyên; testcase đã chỉnh theo mục tiêu an toàn phần mềm, kết quả mong đợi không chứa kết luận cũ. Code có kịch bản cho 29 testcase API; 75 testcase còn lại được ghi ngoài campaign, chưa được xác minh lại. Concolic bổ sung khảo sát TC-ORD-3/4, không làm tăng số ID đã triển khai. 4

2. Hệ thống mục tiêu và dữ liệu 4

3. Phương pháp thực hiện 5

3.1. Blackbox khám phá 5

3.3. Oracle và cách phân loại 5

4. Đặc tả và testcase API 6

5. Kết quả thực nghiệm mới 6

5.1. Đối chiếu theo đặc tả 6

5.2. Luồng blackbox → whitebox 7

6. Thảo luận và hạn chế 7

7. Kết luận 8

Đề tài có blackbox, khảo sát coverage và whitebox concolic có chọn lọc trên xử lý totalAmount của API mô phỏng. Workbook giữ 104 testcase, trong đó 29 có kịch bản API; các phần bổ sung chưa được xác minh đầy đủ. Trọng tâm là liên kết yêu cầu, input, nhánh và bằng chứng. Không tuyên bố symbolic toàn bộ JavaScript, xác nhận lỗi bộ nhớ production hay chứng minh hệ thống không còn lỗ hổng. 8

Tài liệu và mã nguồn tham chiếu 8

Phụ lục: tái hiện và công khai hỗ trợ 8

Phụ lục cập nhật: whitebox concolic và thực nghiệm Chương 4 10

1. Giới thiệu và phạm vi

Đề tài áp dụng fuzzing cho API bán hàng mô phỏng: blackbox khám phá input bất thường; coverage-guided chạy lại, khảo sát lân cận và chọn input mở thêm dòng; whitebox concolic có giới hạn dùng trace nhánh totalAmount và Z3 để sinh input, xác nhận qua API. Các phần được đánh giá riêng theo bằng chứng thực thi.

Năm module gồm đăng nhập, tạo đơn, trạng thái đơn, người dùng và sản phẩm. Oracle phát hiện vi phạm quyền, toàn vẹn dữ liệu, trạng thái và xử lý input. Bộ chạy API chưa thực thi timing, bộ đếm, concurrency/version, CTL hoặc các mẫu C/Python; SMT hiện dùng riêng trong concolic có chọn lọc, không xác minh toàn bộ mô hình Excel.

Bộ đặc tả giữ 10 sheet/file và 104 testcase: AUTH 19, ORD 21, OS 22, USR 21, PRD 21. ID được giữ nguyên; testcase đã chỉnh theo mục tiêu an toàn phần mềm, kết quả mong đợi không chứa kết luận cũ. Code có kịch bản cho 29 testcase API; 75 testcase còn lại được ghi ngoài campaign, chưa được xác minh lại. Concolic bổ sung khảo sát TC-ORD-3/4, không làm tăng số ID đã triển khai.

2. Hệ thống mục tiêu và dữ liệu

Đối tượng là node_harness: ứng dụng Express dùng datastore trong bộ nhớ và dịch vụ mô phỏng. Request thực sự được xử lý; dữ liệu và kết quả không phải bảng kết quả dựng sẵn. Không suy ra mức an toàn của production hoặc MongoDB thật.

Dữ liệu nền là tài khoản/sản phẩm mẫu trong models.js. Seed request được lưu trong seed_corpus/*.json. testcases.py chứa các kịch bản API có ID liên kết workbook. Seed ngẫu nhiên giúp tái tạo chuỗi lựa chọn mutation trong cùng phiên bản code và thứ tự gọi.

```
models.js + seed_corpus
          ↓
blackbox: mutation → HTTP → input có triệu chứng
          ↓
whitebox: replay → lân cận → coverage mutation
          ↓
input + response + loại lỗi + coverage
```

Đăng nhập được blackbox thử trực tiếp, đồng thời phục vụ setup/token của whitebox. Whitebox hiện khảo sát 6 target về đơn hàng, trạng thái, hồ sơ bản thân/người khác và tạo/sửa sản phẩm; chưa khảo sát target login riêng.

3. Phương pháp thực hiện

3.1. Blackbox khám phá

blackbox_fuzzer.py thiết lập trạng thái ban đầu, lấy seed, chọn API rồi gọi mutate_json(). Mutation gồm thay giá trị, xóa trường, đổi kiểu, sửa trường lồng nhau và nhân bản phần tử mảng. Fuzzer gửi HTTP và lưu các lượt HTTP 5xx, timeout hoặc vi phạm oracle. Có chuỗi dạng SQL/XSS trong từ điển không có nghĩa đã xác nhận SQL injection/XSS.

Blackbox giữ dữ liệu suốt chiến dịch; request trước có thể ảnh hưởng request sau. Công cụ lưu target, input, had_auth và bằng chứng. Danh mục sản phẩm được kiểm tra trước khi dùng để đối chiếu giá; danh mục hỏng được ghi riêng, không làm công cụ dừng vì truy cập price bị thiếu.

3.2. Khảo sát coverage và whitebox concolic

focus_blackbox() nhận report blackbox và chọn tối đa max-cases ứng viên có target và ngữ cảnh auth được hỗ trợ. Nó không tự đoán auth bị thiếu trong báo cáo cũ. Chế độ thực hiện ba pha:

Pha	Việc thực hiện
exact_replay	Chạy lại nguyên input và trạng thái bật/tắt token
neighbor_probe	Thử x−1, x+1, 0 hoặc chuỗi ngắn/dài hơn một ký tự
coverage_mutation	Đột biến tiếp, giữ input mở được dòng code mới

Mỗi input chạy trong Node mới. Coverage thu bằng Node Inspector/v8-to-istanbul trong index.js, có cả setup. Chế độ khảo sát cũ là coverage-guided/greybox, không phải symbolic execution. Chế độ --concolic bổ sung mô hình thủ công cho các điều kiện kiểu totalAmount; trace nhánh thật, phủ định tiền tố bằng Z3 và chạy API xác nhận. Chi tiết và số liệu nằm trong phụ lục cập nhật concolic.

Các điểm lân cận chưa xác định đầy đủ vùng lỗi hoặc biên nhỏ nhất/lớn nhất. Không tái hiện được trên dữ liệu sạch chưa đủ bác bỏ lỗi phụ thuộc chuỗi. Chưa tự khôi phục toàn bộ lịch sử thao tác blackbox.

3.3. Oracle và cách phân loại

Oracle kiểm tra response, giá danh mục, dữ liệu lưu, quyền và vòng đời đơn. Ví dụ, tạo đơn bằng giá client tự khớp vẫn sai nếu tổng lưu khác giá catalog; cập nhật hồ sơ thành công không có token là dấu hiệu thiếu bảo vệ.

Bằng chứng	Kết luận được phép

Vi phạm oracle	Finding nghiệp vụ/bảo mật trên input đó
HTTP 5xx	Lỗi phản hồi ứng dụng, chưa phải crash tiến trình
TIMEOUT	Nghi vấn hang; còn phải loại trừ setup/kết nối/tải máy
Lỗi giao thức hoặc setup	Lỗi bộ chạy, chưa kết luận lỗi ứng dụng
Không thấy finding	Chưa tìm được trên mẫu này, không chứng minh an toàn

Không có instrumentation native trong phạm vi mới nên không công bố lỗi bộ nhớ runtime đã xác nhận. Các chỉ tiêu chưa đo được giữ null, không đổi thành số 0 để khẳng định không có lỗi.

4. Đặc tả và testcase API

Mỗi workbook giữ đủ ThongTin, YeuCauChucNang, YeuCauPhiChucNang, RangBuoc, MoHinhTrangThai, ThuocTinhAnToan, LoaiTruLanNhau, HamSinhMa, KiemTraSoHoc và TestCase. ID và schema được giữ nguyên; nội dung testcase đã chỉnh về an toàn đầu vào, quyền, trạng thái, ngoại lệ và bằng chứng lỗi. Các mẫu bộ nhớ là mô hình độc lập, cần công cụ xác nhận; không tự là lỗi Node.js.

Login giữ 19 testcase trong Excel, trong đó bộ chạy API hiện thực thi 5 testcase: thiếu trường, sai kiểu, phản hồi tiết lộ tài khoản, role trong token và email dài. Các kịch bản API ở module khác kiểm tra cấu trúc giỏ, trường tiền, giá catalog, quyền sửa dữ liệu, ID/enum, mật khẩu, ID sản phẩm trùng và trạng thái kết thúc của đơn.

Bộ chạy theo đặc tả là công cụ đối chiếu oracle và bằng chứng. Với fuzz-cases=8, 29 testcase tạo 59 kịch bản. Không tuyên bố đang thực thi 104 testcase hoặc kiểm chứng mô hình hình thức trong phiên bản hiện tại.

5. Kết quả thực nghiệm mới

5.1. Đối chiếu theo đặc tả

Nguồn hiện tại: results/spec_safety_current/report.json; seed=7, fuzz-cases=8. Snapshot gồm 104 testcase trong 10 sheet/workbook; campaign thực thi 29 testcase API: 9 PASS, 20 FAIL; 59 kịch bản: 19 PASS, 40 FAIL, 0 ERROR. Có 75 testcase ngoài campaign được ghi lý do, không gán PASS/FAIL. Các điều kiện bổ sung trong đặc tả chưa được	Số testcase	PASS	FAIL

mọi oracle tự động bao phủ đầy đủ. FAIL không phải số lỗi gốc độc lập. Module			
AUTH	5	4	1
ORD	8	2	6
OS	6	1	5
USR	5	1	4
PRD	5	1	4

Ví dụ TC-ORD-18: catalog có productId=1 giá 200000; client gửi price=1, quantity=2, totalAmount=2 và không có phí/giảm giá. Server chấp nhận lưu tổng 2 thay vì từ chối hoặc tính lại 400000. Đây là phản ví dụ nghiệp vụ, không phải bằng chứng buffer overflow.

Ví dụ TC-OS-21: đơn được hủy thành công nhưng request chuyển sang đang giao vẫn được chấp nhận. Oracle đối chiếu response và trạng thái cuối. Các lỗi cần được phân tích và sửa ở ứng dụng mục tiêu; sửa công cụ không tự sửa các lỗi đó.

5.2. Luồng blackbox → whitebox

Số liệu trong bảng sau là lịch sử tại results/api_fuzz_29/report.json: blackbox 21 input và khảo sát coverage 16 lượt. Kết quả demo mới có concolic nằm ở results/demo_chuong4_final/ và phụ lục cập nhật, không trộn với bảng lịch sử. Giai đoạn	Input	HTTP 5xx	Finding nghiệp vụ/bảo mật
Blackbox	21	1	12
Whitebox khảo sát	16	6	6

Đây là số lượt, chưa phải số lỗ hổng. Ngân sách nhỏ dùng kiểm tra hoạt động của luồng, chưa chứng minh hiệu quả thống kê. Kết quả 48 testcase/96 kịch bản, 104 thiết kế và CTL/SMT của phiên bản cũ không được gán vào phạm vi mới.

6. Thảo luận và hạn chế

Bộ công cụ hiện có 28 bài hồi quy đạt. Năm bài mới kiểm tra phủ định tiền tố, UNSAT, phân biệt missing/null, từ chối mô hình sai và so khớp 8 đại diện input với nhánh API thực đo. Kiểm tra hồi quy công cụ không đồng nghĩa toàn bộ 104 testcase đã được thực thi.

Chưa đo rate limiting, heap/cache, lỗi cấp phát native hoặc ảnh hưởng trên MongoDB thật. Report concolic có thời gian và tốc độ chiến dịch gồm setup/solver/replay, không phải tốc độ thuần của ứng dụng. Coverage gồm setup và chưa đo branch/edge coverage toàn hệ thống. Target login chưa thuộc khảo sát coverage. HTTP 500 không chứng minh crash/hang.

Đề xuất khắc phục ở ứng dụng là kiểm tra kiểu/giá trị trước khi lưu, tính giá theo catalog, bảo vệ endpoint/whitelist và cưỡng chế chuyển trạng thái. Chưa trình bày các đề xuất này là bản vá đã triển khai. Nếu cần nghiên cứu bộ nhớ runtime, phải mở một phạm vi thực nghiệm riêng có thành phần native và công cụ đo phù hợp.

7. Kết luận

Đề tài có blackbox, khảo sát coverage và whitebox concolic có chọn lọc trên xử lý totalAmount của API mô phỏng. Workbook giữ 104 testcase, trong đó 29 có kịch bản API; các phần bổ sung chưa được xác minh đầy đủ. Trọng tâm là liên kết yêu cầu, input, nhánh và bằng chứng. Không tuyên bố symbolic toàn bộ JavaScript, xác nhận lỗi bộ nhớ production hay chứng minh hệ thống không còn lỗ hổng.

Tài liệu và mã nguồn tham chiếu

1. Tài liệu hướng dẫn báo cáo bài tập lớn của giảng viên do người học cung cấp; cấu trúc trình bày được tham khảo, phạm vi nội dung đã được thu gọn theo lựa chọn hiện tại.

2. Mã nguồn trong fuzzers/, node_harness/ và helper server/concurrency.js; phiên bản kiểm thử được ghi bằng source_sha256 trong report.json.

3. Đặc tả và nguồn dữ liệu: BaoCaoATPM/Module_*_spec.xlsx, seed_corpus/*.json; hash workbook nằm trong spec_snapshot.json của campaign.

4. Node.js Inspector API: https://nodejs.org/api/inspector.html; MITRE CWE về phân quyền/validation/mật khẩu đã dùng trong bản trước chỉ là nguồn khái niệm, không tự gán CVE.

Phụ lục: tái hiện và công khai hỗ trợ

```
python -m pip install -r fuzzing-security-testing/requirements.txt
npm ci --prefix fuzzing-security-testing/node_harness
python fuzzing-security-testing/fuzzers/run_fuzzing.py --iterations 21 --iterations-per-case 8
--max-cases 2 --seed 7 --output fuzzing-security-testing/results/new_main
python fuzzing-security-testing/fuzzers/run_ch4.py --seed 7 --fuzz-cases 8 --output fuzzing-
security-testing/results/new_api
```

```
python -m unittest discover -s fuzzing-security-testing/tests -p "test_*.py"
```

Báo cáo, source và kết quả cần nộp cùng cấu trúc đường dẫn; harness dùng helper server/concurrency.js ở thư mục gốc. Nếu chưa có ứng viên blackbox phù hợp, bộ chạy ghi SKIPPED_NO_CANDIDATE thay vì tạo kết quả whitebox giả.

AI hỗ trợ rà soát và tổ chức đặc tả, chỉnh sửa code/testcase, điều phối chạy, phân tích bằng chứng và biên soạn báo cáo. Người học cần tự hiểu mã nguồn, kiểm tra kết luận và chịu trách nhiệm với bài nộp.

Phụ lục cập nhật: whitebox concolic và thực nghiệm Chương 4

Liên hệ tài liệu: Chương 1 phần 1, trang 41 và 49: lỗi đầu vào, lỗi xử lý ngoại lệ và logic nghiệp vụ thuộc an toàn phần mềm. Chương 4 phần 1, trang 10–11 và 41: oracle/invariant và vòng lặp coverage. Chương 4 phần 2, trang 7–9, 45 và 53–54: trace, phủ định path condition, SMT và UNSAT.

Đối tượng concolic: POST /api/orders → OrderModel.save → castNumberOrThrow(totalAmount). Nhánh hiện hữu được instrument tại models.js; không cấy thêm lỗi vào API. Biến total_kind thuộc 0..7 mô tả tám đại diện number, numeric string, invalid string, object, null, missing, boolean, array. Các trường khác giữ seed hợp lệ. Đây là mô hình kiểu có chọn lọc, không symbolic mọi giá trị hoặc toàn bộ JavaScript.

Quy trình: chạy seed 400000; lấy branch_trace thực; giữ điều kiện tiền tố, phủ định một điều kiện, bỏ hậu tố cũ; Z3 trả SAT/model hoặc UNSAT/unknown; giải mã model thành JSON, gửi lại API và kiểm tra tiền tố nhánh. validate_trace phát hiện mô hình lệch, không ghi kết quả thành công giả. Input có vi phạm được chạy lại trong Node mới.

Ví dụ SAT: phủ định order.total.required từ false sang true; solver chọn null hoặc missing. API được đo nhận input đó qua HTTP. Ví dụ UNSAT: required=false yêu cầu không null/missing, nhưng cast.empty=true yêu cầu null/missing. Mâu thuẫn chỉ có ý nghĩa trong tiền tố và miền đã mã hóa.

Kết quả results/demo_chuong4_final/, seed 7: blackbox 42 input, 1 HTTP 5xx và 22 finding occurrences; khảo sát coverage 16 lượt, 6 HTTP 5xx và 6 finding occurrences. Concolic chạy 5 input mới và 4 replay: 2 input HTTP 5xx, 4 input vi phạm oracle, 1 truy vấn UNSAT. Bốn input vi phạm tái hiện cùng mã HTTP, trace và số đơn được tạo. Chấp nhận numeric string/boolean vi phạm hợp đồng API yêu cầu số, chưa tự chứng minh thiệt hại hoặc khả năng khai thác.

Kết quả các giai đoạn không cộng thành số lỗ hổng duy nhất. Đã lưu công thức, model, input, response, trace, dữ liệu thay đổi và source hash trong report/evidence. Bộ 28 kiểm tra hồi quy

đạt, gồm đối chiếu cả 8 đại diện với nhánh Node. Campaign concolic chỉ cần một số đại diện để mở các đường khác nhau, không tuyên bố thử mọi input JSON.

Lệnh demo từ thư mục gốc:

python fuzzing-security-testing/fuzzers/run_fuzzing.py --iterations 42 --iterations-per-case 8 --max-cases 2 --seed 7 --concolic

Giới hạn: datastore mô phỏng; mô hình thủ công; không tự kích hoạt symbolic khi coverage plateau; không trao đổi corpus hai chiều; chưa giảm input lỗi, dedup theo stack hoặc đo lỗi bộ nhớ bằng sanitizer. Mỗi input concolic dùng seed hợp lệ, không khôi phục toàn bộ trạng thái blackbox. Không tuyên bố whitebox tổng quát hay hiệu quả cao hơn enumeration trong miền tám đại diện.

Hướng dẫn trình bày, hàm cần giải thích và câu hỏi thường gặp: fuzzing-security-testing/TRINH_BAY_TIEN_DO.md.